

OptiDBX Technical Report

OptiDBX: Safe, Measurement-Grounded Online Auto-Tuning of PostgreSQL Query Parallelism

A design description and empirical study

- **Project:** OptiDBX — Autotuner & Integration
 - **Platform:** PostgreSQL 16, single-host prototype
 - **Document type:** Technical report
-

Abstract

- Online database tuners must decide *while the system is live* whether an applied change actually helped, and must do so without destabilizing the database or being misled by noisy measurements.
 - This report presents **OptiDBX**, a safe online auto-tuner for a single PostgreSQL parallelism knob (`max_parallel_workers_per_gather`), and an empirical study of its behavior.
 - **Contribution 1 — methodology:** KEEP/ROLLBACK decisions are grounded in client-observed, warm-up-excluded measurements of the *owned* workload (not database-wide server counters), evaluated under an explicit net-benefit criterion with a hard latency-regression guard and verified rollback.
 - **Contribution 2 — empirical characterization:**
 - On a well-provisioned host, reducing query parallelism yields no measurable benefit and the tuner correctly takes **no action**.
 - Under genuine CPU oversubscription — the condition the action targets — the tuner keeps beneficial reductions, reaching a **50% keep rate**, with every kept action backed by a measured tail-latency improvement.
 - **Contribution 3 — failure modes:** three issues that silently defeat naive online tuners are documented — cold-start measurement inflation, a stale-timestamp defect that prevents any automatic action, and monitor starvation under saturation.
 - **Scope:** single-knob prototype; positioned as an engineering and measurement study, not a novel tuning algorithm.
-

1. Introduction

- **Problem context:**
 - Relational DBMSs expose hundreds of parameters whose optimal values depend on workload, data, and hardware.
 - Manual tuning is labor-intensive and error-prone, motivating automatic configuration.
- **Offline vs online tuning:**

- Most prior work is *offline*: explore configurations on a replica/session, then install one for production.
 - *Online* tuning observes a live workload, applies a change, and must immediately judge its effect.
 - **Why online decisions are hard:**
 - Keeping a harmful change degrades production; reverting a helpful one forfeits benefit and churns the system.
 - The tuner must be able to undo any change with certainty and never leave an ambiguous state.
 - The tuner must not itself become a source of instability.
 - **OptiDBX approach:**
 - Restrict to a single, reversible action (reduce per-gather parallelism by one approved step).
 - Invest complexity in *deciding correctly and acting safely*, not in exploring a large configuration space.
 - **Contributions:**
 - A **safe online action lifecycle**: pre-mutation journaling, verified apply, bounded observation, verified rollback, fail-closed recovery.
 - A **measurement methodology** for online KEEP/ROLLBACK decisions based on owned-workload, warm-up-excluded, client-observed metrics under a net-benefit criterion.
 - An **empirical characterization** of when parallelism reduction helps, including three documented failure modes.
 - **Explicit non-goal:** OptiDBX is not a novel tuning algorithm and does not compete on knob breadth with learned multi-knob tuners.
-

2. Background and Related Work

- **Offline / model-based tuning:**
 - *iTuned* [Duan et al., VLDB 2009] — adaptive sampling with response-surface models.
 - *OtterTune* [Van Aken et al., SIGMOD 2017] — Gaussian-process models with cross-session knowledge transfer.
 - *BestConfig* [Zhu et al., SoCC 2017] — automatic configuration search under resource constraints.
- **Learning-based tuning:**
 - *CDBTune* [Zhang et al., SIGMOD 2019] — deep reinforcement learning, end-to-end cloud DB tuning.
 - *QTune* [Li et al., VLDB 2019] — query-aware RL tuning.
 - *UDO* [Wang et al., VLDB 2021] — joint optimization over heterogeneous knobs.
- **Autonomous systems vision:**
 - *Self-Driving DBMS* [Pavlo et al., CIDR 2017] — forecast, plan, and act autonomously.
 - Commercial auto-advisors and self-tuning features (Oracle, SQL Server, cloud-managed engines).
- **Positioning of OptiDBX:**
 - Deliberately narrow: single knob, heuristic detector — not a learned multi-knob model.

- Orthogonal focus: the **online decision and safety layer** (trustworthy live measurement + safe apply/rollback).
 - Prior tuning work largely assumes configuration effects can be measured cleanly; this report shows that assumption is fragile online and quantifies the consequences.
-

3. System Design

- **Closed monitoring loop:**

- workload → OS + DB telemetry → detection → recommendation → (display | safe apply) → bounded observation → KEEP/ROLLBACK → cooldown → monitor.
- Strict separation between passive monitoring and active tuning.

3.1 Telemetry

- OS metrics: CPU utilization, memory pressure, disk I/O, context switches.
- DB metrics: query latency, throughput, temporary-file usage, active parallel workers.
- Fixed 5 s sampling interval.
- Detection-time DB latency/throughput derived from `pg_stat_statements` / `pg_stat_database` (decision-time limitations addressed in Section 4).
- Samples with excessive OS/DB timestamp skew, missing intervals, or counter resets are **rejected, not imputed**.

3.2 Detection

- A CPU/parallelism bottleneck is confirmed only after **three consecutive** problematic intervals (≈ 15 s of sustained behavior).
- Each interval must satisfy a conjunction: elevated CPU, elevated context-switch rate, high active parallel-worker count, elevated query latency.
- Purpose: avoid reacting to transient spikes.
- Thresholds are externalized in shared config and treated as calibratable, not universal.

3.3 Action model and safe lifecycle

- **Action:** `REDUCE_DB_PARALLELISM` — lowers `max_parallel_workers_per_gather` one step within an approved whitelist (1, 2, 4, 6, 8).
- **Preconditions:** only an explicitly bound, application-owned workload session is tunable; external sessions are monitor-only; one action at a time.
- **Journal before mutation:** recovery record (target, before-state, scope) is fsync-ed and an audit row persisted *before* any change; if either write fails, no mutation occurs.
- **Verified apply:** new value is set, then read back for confirmation.
- **Bounded observation:** fixed 30 s window; first interval excluded to avoid transition effects.
- **Decision:** KEEP or ROLLBACK per Section 4.
- **Verified rollback / cooldown:** on ROLLBACK the original value is restored and read back; a 30 s cooldown follows either outcome.

- **Fail-closed recovery:** an unresolved rollback marks the system `ROLLBACK_FAILED`, retains original sessions, and blocks further actions until an operator intervenes; after a crash the on-disk journal remains authoritative and cannot be impersonated by a new connection.

3.4 Avoiding repeated ineffective actions

- Performance-based rollbacks are recorded against a coarse workload-condition signature (parallel-worker band, CPU band).
 - The detector suppresses the same reduction under comparable conditions until a TTL elapses or conditions change materially.
 - Effect: prevents repeated ineffective attempts while allowing reconsideration on workload shift.
-

4. Measurement Methodology

- **Central design decision:** *how* the KEEP/ROLLBACK question is answered. Three principles apply.

4.1 Measure the owned workload, not the database

- Database-wide counters are always available but unsuitable as a *decision* signal:
 - `pg_stat_database` throughput counts all transactions (including the monitor's own commits and unrelated activity) and is insensitive to the speed of a few large analytical queries.
 - `pg_stat_statements` mean statement time aggregates across statement types at coarse granularity.
- OptiDBX instead measures the *owned* workload directly:
 - Each query from bound workload sessions is timed at the client around its round trip.
 - Per-query latencies and completion counts are summarized as median/p95 latency and queries-per-second (QPS).
- Database-wide telemetry is retained only as a fallback for the monitor-only case (no owned workload, no automatic action possible).

4.2 Exclude warm-up from the baseline

- A baseline captured while a workload is still ramping (cold caches, connection warm-up) understates baseline performance and inflates the apparent post-change improvement.
- Requirements before a change is applied:
 - The owned workload must have been warm for a configured interval.
 - An initial warm-up slice is excluded.
 - A complete, warm baseline window with a minimum number of completed queries must exist.
- Omitting this step produces large spurious improvements (quantified in Section 6.2).

4.3 Net-benefit decision criterion

- Rationale: a parallelism reduction trades single-query latency against aggregate behavior; a single-axis rule is inadequate.

- **net_benefit (default):**
 - Keep when a weighted combination of throughput gain and latency improvement exceeds a threshold.
 - Hard guards: never keep if tail latency regresses beyond a cap, throughput regresses beyond a cap, or error rate increases.
 - OS resource degradation (CPU/memory/disk) is an independent veto.
 - **latency_first / throughput_first:** provided for comparison.
 - Every decision records its source (owned vs telemetry), the measured percentage changes, and the policy — fully auditable.
-

5. Experimental Setup

- **Host:** 16 logical CPU cores; PostgreSQL 16 on Ubuntu (WSL2) under Windows 11.
 - **Dataset:** pgbench scale factor 10 (~1M `pgbench_accounts` rows, ≈150 MB).
 - **Workload:** owned analytical read sessions aggregating over `pgbench_accounts`.
 - Concurrency profiles: 1 / 4 / 10 sessions (LOW / MEDIUM / HIGH).
 - Two query variants: scan-dominated aggregate (default) and compute-intensive aggregate (CPU-bound regime, Section 6.4).
 - **Server pool:** default `max_parallel_workers = 8` unless stated otherwise.
 - **Tuner configuration:**
 - 5 s sampling; three-reading confirmation.
 - 15 s warm-up + 25 s warm baseline; 30 s observation (first interval excluded); 30 s cooldown.
 - Minimum 30 completed owned queries per window.
 - `net_benefit` policy: 5% net-improvement threshold; 10% hard latency-regression cap.
 - **Measurement:** client-observed per-query latency (median, p95) and QPS, warm-up excluded, over fixed windows; sweeps interleaved to reduce order/cache effects.
 - **Experiment categories:**
 - **Ground-truth sweeps** at fixed parallelism (no tuner) — establish whether reduction *can* help.
 - **Closed-loop tuner runs** — measure how often the tuner keeps a beneficial change (keep rate).
-

6. Results

6.1 Well-provisioned host: no benefit from reduction

- Fixed-parallelism sweep, HIGH profile (10 sessions), warm-up excluded, default 8-worker pool:

per_gather	QPS	p95 latency (ms)
8	11.81	1512
4	11.65	1546
2	12.09	1478
1	12.15	1401

- Throughput spread $\approx 4\%$ across the full range — within run-to-run noise.
- Interleaved A/B (parallelism 2 vs 1): +1.0% QPS, -5.9% p95 — also within noise.
- **Structural cause:** 16 cores with a shared pool capped at 8 workers $\Rightarrow$ never CPU-oversubscribed by parallel workers $\Rightarrow$ no contention to relieve.
- **Plan inspection:** the workload requests only 3 workers regardless of the per-gather setting; dominated by a parallel sequential scan over cached data (≈ 119 ms) $\Rightarrow$ memory-bandwidth bound, not CPU bound.
- Raising the shared pool to 48 left the sweep flat (the planner does not spawn more workers than query cost warrants).
- **Finding:** correct action is *no action*; the existing configuration is already appropriate.

6.2 Cold-start measurement artifact

- Early configuration measured the baseline immediately after workload start (cold caches/connections).
- Observed apparent effect: reductions appeared to nearly double throughput ($\approx 6.4 \rightarrow \approx 12$ QPS).
- Warm-up-excluded steady-state comparison: true baseline ≈ 11 QPS $\approx$ post-change value.
- **Conclusion:** the apparent $\approx 2\times$ gain was an artifact of a cold baseline vs a warm observation $\Rightarrow$ motivates warm-up exclusion (Section 4.2).
- **Generalization:** any online tuner comparing pre/post windows without controlling warm-up is exposed to this bias.

6.3 Defect preventing all automatic action

- The auto-approval path submitted a recommendation whose timestamp was fixed at first detection, while the baseline sample window advanced each interval.
- The apply-time continuity check then rejected the recommendation as discontinuous on every subsequent interval.
- Consequence: a *stable* recommendation could never be applied automatically; only recommendations regenerated after a transient reset were applied, and then only by timing luck.
- **Fix:** refresh the recommendation timestamp to the current interval at approval time (as the manual path already did).
- **Significance:** consistent with the system's prior inability to ever record a KEEP; the hardest part of online tuning is often the decision/apply plumbing, not the policy.

6.4 Under CPU oversubscription, reduction helps

- Regime constructed with a CPU-bound query, forced parallelism, and an enlarged pool so per-gather parallelism genuinely controls CPU-bound worker count.
- Fixed-parallelism sweep (6 sessions, warm-up excluded):

per_gather	QPS	p95 latency (ms)
8	23.05	606
4	22.52	329
2	23.92	300
1	23.12	290

- Throughput flat; **tail latency roughly halves** as parallelism is reduced (fewer workers relieve CPU oversubscription while the same work completes).
- This is a genuine, measurable benefit and precisely the net-benefit signal (latency gain at stable throughput) the decision criterion targets.

6.5 Closed-loop keep rate

- Setup: CPU oversubscription created by pinning PostgreSQL to a subset of cores (workload oversubscribes CPU; monitor retains headroom — Section 6.6); MEDIUM profile; six cycles each; each cycle starts from the same initial parallelism and records the first completed action.
- **Moderate oversubscription (PostgreSQL on 12 cores):**

Cycle	Δ QPS	Δ p95	Net	Outcome
0	+1.9%	−3.3%	+3.6%	ROLLBACK
1	—	—	—	no action
2	−0.3%	+1.8%	−1.2%	ROLLBACK
3	+4.6%	−8.5%	+8.8%	KEEP
4	+2.7%	−4.3%	+4.8%	ROLLBACK
5	+1.3%	−3.3%	+2.9%	ROLLBACK

- Keep rate among completed actions: **20% (1/5)**.
- Every completed action gave a small real improvement; only one cleared the 5% net threshold ⇒ correctly conservative.
- **Higher oversubscription (PostgreSQL on 8 cores):**

Cycle	Δ QPS	Δ p95	Net	Outcome
0	+4.3%	−8.8%	+8.6%	KEEP
1	+0.4%	−0.4%	+0.6%	ROLLBACK
2	+4.5%	−6.6%	+7.8%	KEEP
3	+3.1%	−1.3%	+3.8%	ROLLBACK
4	+4.5%	−5.5%	+7.3%	KEEP
5	−7.4%	+8.7%	−11.8%	ROLLBACK

- Keep rate among completed actions: **50% (3/6)**.
- Every KEEP corresponds to a measured tail-latency improvement of 5.5–8.8%.
- Every ROLLBACK is a marginal or genuinely worse outcome; cycle 5 (worse on both axes) was correctly reverted.
- No outcome was relabeled and no run discarded.

6.6 Monitor starvation under saturation

- At near-100% CPU, the monitoring loop shares cores with the saturating workload and cannot reliably hold its 5 s cadence.
- The apply-time baseline continuity check then intermittently rejects otherwise-valid actions.
- **Constraint conflict:** detection requires system CPU above a threshold, while stable monitoring requires spare CPU — the two conflict on a busy host.
- Reserving CPU for the monitor (core pinning in these experiments) restores reliable cadence and enables the keep-rate results.
- **General lesson:** the monitor can be starved by the very load it is meant to observe.

7. Discussion

- **Measurement dominates policy:**
 - The move from *never keeping* to a 50% keep rate came from correct measurement and apply plumbing — owned-workload signals, warm-up exclusion, and the timestamp-refresh fix — not a smarter policy.
 - Two of the three decisive issues (Sections 6.2, 6.3) corrupt results silently rather than failing loudly — the most dangerous class of bug for an autonomous system.
- **"No action" is a first-class correct outcome:**
 - On a well-provisioned host the action provides no benefit and the tuner does nothing.
 - Reporting this honestly (rather than manufacturing activity) is a correctness property.
 - Benefit appears only under genuine CPU oversubscription, where the tuner captures it.
- **Online self-monitoring is a design constraint, not an afterthought:**
 - A tuner that degrades exactly when the system is most stressed is of limited use.
 - Monitoring resource isolation must be part of the design.

8. Limitations and Threats to Validity

- **Narrow scope:** one parallelism knob, analytical read workloads, single host; does not generalize to multi-knob tuning, mixed OLTP/OLAP, or diverse hardware.
 - **Constructed positive regime:** the 50% keep rate was obtained under a deliberately over-parallelized, CPU-constrained configuration — realistic but constructed, not sampled from production traces; read as *"under the conditions the action targets, the tuner captures the available benefit."*
 - **Small samples:** keep rates over a handful of cycles without confidence intervals — indicative, not statistically conclusive.
 - **No baseline comparison:** not evaluated against existing tuners or DBA heuristics.
 - **Prototype scope:** single local process; no authentication, multi-node, or multi-tenant coordination.
 - **Overall positioning:** an engineering and measurement study, not a novel tuning-algorithm contribution.
-

9. Future Work

- **Breadth:** standard benchmarks (TPC-C, TPC-H, TPC-DS, YCSB) and real traces across multiple hardware profiles, with confidence intervals.
 - **Baselines:** direct comparison against default configuration, DBA heuristics, and a learned tuner.
 - **Method:** adaptive thresholds and multi-knob extension while preserving the safety and measurement layer.
 - **Monitoring isolation:** explicit, portable mechanism (cgroups/CPU affinity) rather than manual reservation.
 - **Decision procedure:** statistical (e.g., sequential testing) in place of the fixed-window net-benefit threshold.
-

10. Conclusion

- The difficulty of online DB auto-tuning lies less in choosing a configuration than in **safely applying a live change and honestly measuring its effect.**
 - Grounding KEEP/ROLLBACK in warm-up-excluded, owned-workload measurements under a net-benefit criterion, with a fail-closed apply/rollback lifecycle, moved a single-knob parallelism tuner from never retaining a beneficial change to a **50% keep rate** under the conditions the action targets — while correctly taking no action when no benefit exists.
 - The study surfaces three broadly relevant failure modes: cold-start measurement inflation, a stale-recommendation apply defect, and monitor starvation under saturation.
 - The work is a prototype and methodology study; extending it into a general, evaluated tuning system is future work.
-

References

1. S. Duan, V. Thummala, S. Babu. *Tuning Database Configuration Parameters with iTuned*. VLDB, 2009.
2. D. Van Aken, A. Pavlo, G. J. Gordon, B. Zhang. *Automatic Database Management System Tuning Through Large-scale Machine Learning*. SIGMOD, 2017.
3. Y. Zhu et al. *BestConfig: Tapping the Performance Potential of Systems via Automatic Configuration Tuning*. SoCC, 2017.
4. A. Pavlo et al. *Self-Driving Database Management Systems*. CIDR, 2017.
5. J. Zhang et al. *An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning (CDBTune)*. SIGMOD, 2019.
6. G. Li, X. Zhou, S. Li, B. Gao. *QTune: A Query-Aware Database Tuning System with Deep Reinforcement Learning*. VLDB, 2019.
7. J. Wang, I. Trummer, D. Basu. *UDO: Universal Database Optimization using Reinforcement Learning*. VLDB, 2021.
8. The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation — Parallel Query; Cumulative Statistics System*.